

The Pathfinder engine

Pathfinder, github.com/vd1/pathfinder

12 September 2026, revised 13 September

Abstract

1 Introduction

The pathfinder program. Pathfinder takes two corpora of scientific papers (concretely arxiv deposits), Q and P , and, using agentic methods, looks for pairs (q, p) in $Q \times P$ with enough of an interesting connexion to try and elaborate it as a pre-publication. Ideally, the pre-publication satisfies the usual constraints of correctness, novelty, relevance, readability, and exposition of related work in the literature.

In some cases, the input tuple may be supplemented with a seed note (provided by the agent that scanned and scored the pair in the first place) that is a paragraph describing the putative connexion.

This type of task is commonly carried out by human experts. It happens through seminars that the seed of an idea is provided by someone in the audience, and such connexions are made, and, in favourable circumstances, further pursued all the way to a publication. However, by using modern agents (which are competent experts on every corpus, and always “willing” to contribute), one can do it faster, cheaper, and, therefore, at a larger scale. We have ran it on sets of paper of size about a hundred each for a reasonable token budget.

A small epistemologic aside: Pathfinder is about scientific creativity, (cite Swanson, Cedric Lions, and other refs on creativity by analogy) less about closing conjectures, a task at which modern agents have become so good. Whether this particular attempt at generative creative input is enough or at all capable of creating new accumulative knowledge remains to be seen.

Object of this note. This note describes in implementable terms a simple agentic process whereby an pathfinder input tuple is progressively elaborated into a research note, and, in favourable cases, into a bona fide pre-publication is generated.

The process runs in two phases.

1. The research phase: researcher agents (typically two) collaborate and produce a research note via rounds of interactions with a verifier agent. The agents’ dialog is recorded on a ledger. The final note can be put on pause by the verifier or declared ready for the next phase.
2. The editing phase: an author agent produces via rounds of interactions with a reviewer agent, a final version of the draft. The final draft can be put on pause or accepted as a pre-publication by the reviewer.

An example, and why reruns differ. It is worth noting that running the research phase again on the same input may lead to a different outcome, possibly a draft expounding on a different idea.

Consider the following (q, p, c) input triplet (where c was written by the scanning agent):

$q = \text{arXiv:2608.29130}$ “A Systematic Approach to Mechanism Design with Stochastic Dynamic Stability”, which designs quadratic payments so that self-interested agents settle at the welfare-maximising allocation and learns that equilibrium from sampled feedback

$p = \text{arXiv:2609.04460}$ “Distributed risk-averse optimization via CVaR”, which minimises conditional value-at-risk over a network from noisy function values.

The seed c is: replace q ’s expected-value objectives with CVaR and use p ’s gradient estimator inside q ’s learning loop; does the payment mechanism still work?

The thread was run twice on (q, p, c) . Both runs quickly rejected the literal splice (q needs a best response, p supplies a gradient step) and both ended as DRAFT, but with different results. The first run proved a positive statement: a uniform bound $\sup_x |\tilde{C}_n(x) - C_n(x)| \leq L_n \delta_n + (U_n / \alpha_n) (2\pi / s_n)^{1/2}$ between p ’s smoothed, sampled surrogate and the true CVaR objective, and drew from it welfare and individual-rationality guarantees for the risk-averse mechanism. The second run proved obstructions instead: a two-point example in which the expected objective is strongly concave but its CVaR is constant, so the curvature q ’s uniqueness proof needs can vanish under the substitution, and a coupling example showing that the local risk oracles both papers use cannot determine system-wide tail risk. Each note is correct, each was accepted by a verifier that had not taken part.

Thus, the research phase is a sampler. The same input, rerun, may elicit a different response. Arguably, in the example above, a referee would want both responses in one paper.

1.1 Outline

Below we describe the pipeline as a state machine with transitions. The expected behaviours are:

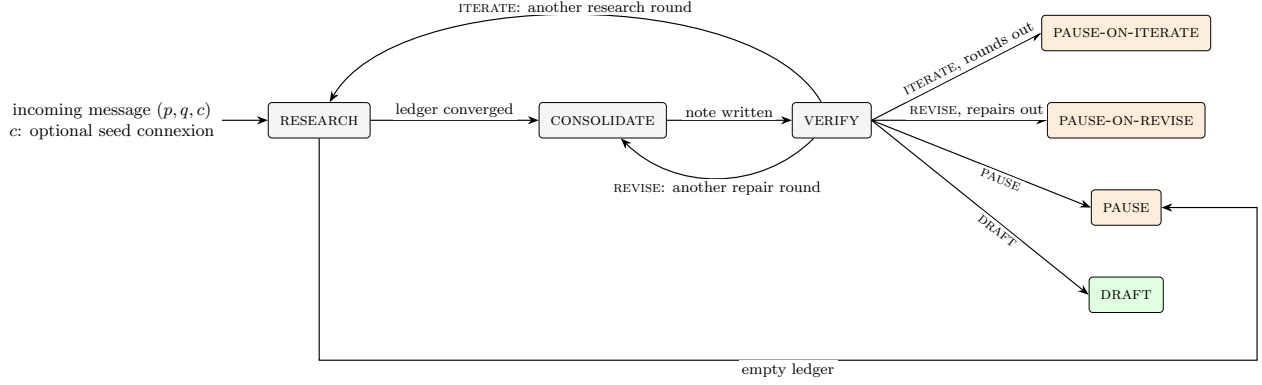
1. budget control: each run of the process has a budget limiting the agents’ resources in time and tokens
2. auditability after the fact: every decision traces to a file and a receipt
3. safe interruptibility and failure recovery: a stop, a model timeout, a crash, an exhausted token budget, or an agent violating their contracts leaves the machine in a named state from which it resumes with minimal waste of resources and work already committed

2 The research phase

The research phase takes the incoming message (q, p, c) and ends in one of four terminal states:

1. DRAFT, the note carries a result that would justify a paper
2. PAUSE, the pair does not yield anything worth further effort now, or there are missing inputs which the agents cannot supply
3. PAUSE-ON-ITERATE and PAUSE-ON-REVISE, the loop named ran out of its budget with the verifier still asking. Each arrow of the diagram is described below, in the order a thread meets them.

Here is a diagram which summarises the different states and transitions. Details are given below.



Research. Upon receiving a triplet (p, q, c) , the runner process creates a specific directory, with the two associated papers p and q and seed c , and launches the duo of researcher agents concurrently with their prompt, and tools (files, shell, code, web search), research allowance (number of rounds, number of model calls per round, total agent time, and total token consumption), and own sub-directory. Each agent sees the two papers, the seed c with its scores and rationale, the whole ledger, its own directory and its partners' directories, and the allowance left for this call. It writes ledger entries and whatever it likes in its own directory. The first prompt, `peer.md`, asks whether there is something publishable to say about this pair; later rounds add the verifier's last verdict. There is no turn-taking: the two agents run concurrently and each spends its calls at its own pace. An agent declares itself done by writing a *ready* entry, which is void as soon as its partner writes anything substantive. Before spending a call, an agent whose ready still stands does not spend it: its loop polls the ledger and waits, because a fresh call would only repeat what it already said; as soon as the partner writes, the call is made. The round ends when both agents' ready entries stand at once, or when the allowance is out. So a round costs what the disagreement costs: agreement after one exchange is one call each, an objection to every entry is the full allowance. (If the round ends with an empty ledger the thread ends in PAUSE.)

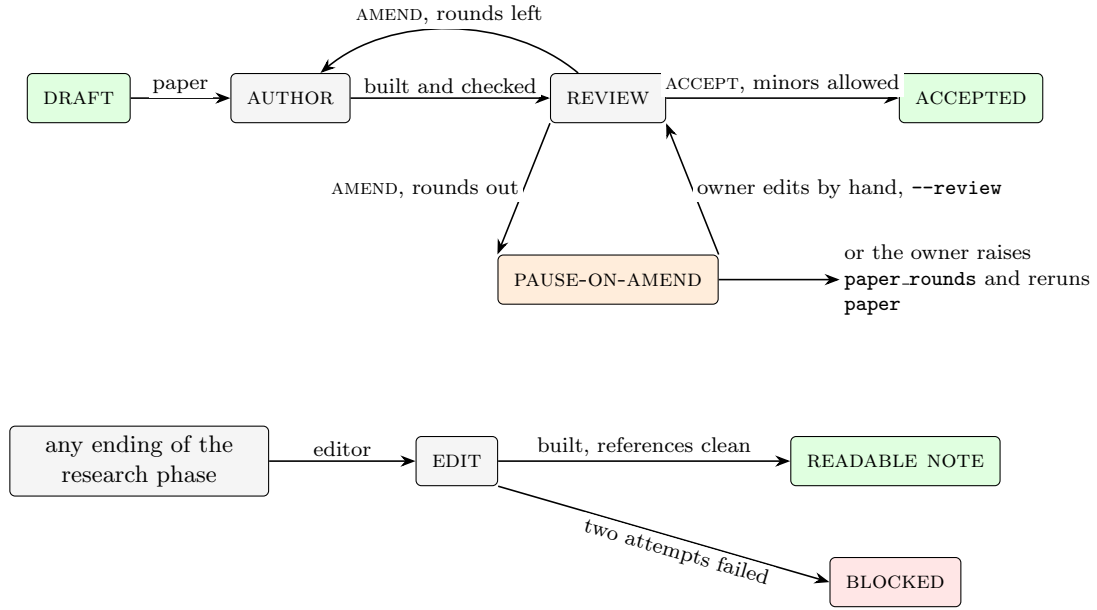
Consolidate. Once researchers are ready, the first researcher (arbitrary) is called once with a new prompt `consolidate.md` and tools, sees the ledger, every directory and the note from preceding round (if any), and writes the note `<pair>.tex`: what the researchers agreed, with every claim pointing at the ledger entries that support it. After an ITERATE the note is appended to, so the history of the thread stays readable; on a REVISE the consolidator also sees the verifier's corrections and rewrites. The call has `consolidate.seconds`; if it produces no note it is rerun once.

Verify. A verifier that has taken no part in the thread is called with `verify.md` and no tools; the two papers, the ledger and the note are given inline, so its only evidence is the record. It writes a verdict file carrying the digest of the note it judged, a review entry in the ledger, and the position. It answers with one of four words. DRAFT: the note contains a supported, non-obvious result about the pair. REVISE: the ledger supports a result but the note misstates it (a dropped result, an overclaim, a missing attribution), and the consolidator can fix it from the ledger without new research; the verdict lists the corrections. ITERATE: a promising line with a specific gap the researchers can close with what they have; the verdict names the one thing to do next. PAUSE: nothing worth further effort now, or a gap that needs data, experiments or access the agents do not have; the verdict says what would be needed so that a human can decide. The verifier is told not to ask twice: if a previous ITERATE asked for something the ledger then shows the researchers could not supply, it must answer PAUSE and name the missing input. The call has `verify.seconds`; an empty reply is retried once; an unreadable verdict is saved beside the position and the thread is BLOCKED.

The iterate loop. On ITERATE the round counter goes up and the researchers are called again, with the verdict in the ledger. The cap is **rounds**, four in the reference campaign; at the cap the thread ends in PAUSE-ON-ITERATE.

The revise loop. On REVISE the repair counter goes up and the consolidator is called again with the corrections. The cap is **repairs**, one; at the cap the thread ends in PAUSE-ON-REVISE. A repair is a rewrite of the note from the same ledger; it costs one call, which is why it has its own word and its own cap rather than spending a research round.

3 The edit phase



The edit phase has two independent branches. The upper one, the *paper*, runs only on a DRAFT: an author agent writes a paper from the note and a reviewer judges it, and the loop ends ACCEPTED or PAUSE-ON-AMEND. The lower one, the *readable note*, runs once for every thread whatever its ending, so that even a paused pair leaves an account a colleague can read without the ledger. Neither branch changes the note or the ledger; the research record is frozen when the research phase ends.

Author. The author agent is called with `author.md`, tools and web search, in the thread’s directory, so it sees everything the researchers left: the two papers, the ledger, the note, the verdicts, every directory. It also sees the previous review’s findings, if this is a later round. It writes `paper/paper.tex`, `paper/references.bib` and `paper/search.md`, the record of the literature searches it ran and what they returned. It is asked for a short paper in the usual shape (abstract, introduction, what the two papers establish, the result, what did not hold, related work, limitations), with every claim traced to the note or the ledger and every reference verified against arXiv or a DOI. The call has `paper_seconds`.

Build and check. Before any reviewer sees the paper the pipeline, with no model, builds it with `latexmk` and checks the bibliography: every cited key exists, every entry is cited, every entry carries a URL, DOI or arXiv identifier, and every arXiv identifier’s title matches the arXiv record. The findings are written to `paper/checks-round-N.txt` and shown to the reviewer; a failed build is a finding too.

Review. A reviewer that has taken no part is called with `review.md` and no tools; it is given inline the paper, the bibliography, the pipeline’s checks, the author’s search record, the note, the ledger and the two papers. It reads the paper as an editor would before sending it to referees and checks that every claim traces to the note or ledger without strengthening it, that the mathematics is correct and the notation defined, that q and p are represented fairly and what is new is separated from what they establish, that no reference looks invented, and that the related-work claim is supported by the recorded searches. It answers `ACCEPT` or `AMEND` with findings by identifier, each marked major or minor. `ACCEPT` may carry minor findings, the kind an author applies while formatting; `AMEND` is reserved for a finding that changes a claim, its scope, its attribution or its correctness, or that a referee would hold against the paper. A reviewer asked for a paper with no findings at all never converges, because every revision opens new minor points; this rule is what lets the loop end. The review is recorded with the digest of the paper it judged. The call has `review_seconds`; the loop’s cap is `paper_rounds`, three; at the cap the paper ends `PAUSE-ON-AMEND`.

The amend loop. On `AMEND` with rounds left, the author is called again with the findings and must say what it did about each. The pipeline builds and checks again, and the reviewer reads the new version.

The owner’s moves. A paper at `PAUSE-ON-AMEND` has two ways forward, both the owner’s. The owner can raise `paper_rounds` and rerun the paper command, which resumes the loop where it stopped. Or the owner can edit the paper by hand and ask for one reviewer round with `paper PAIR --review`: no author call, the paper as it stands, counted as the next round and outside the budget. In the reference campaign one paper went through three such rounds; the reviewer caught a real error in the owner’s own edit before accepting.

Editor. For every thread with an ending, the editor agent is called with `editor.md` and tools but no web search. It sees the two papers and their metadata, the note, the verdicts, the ledger and every directory, and writes `edited/note.tex` with a bibliography: the note rewritten as a short readable paper, in plain terms, for a reader who never saw the ledger, keeping the structure of a paper (what was asked, what was done, what holds, what did not hold, why the thread stopped and what would restart it). The pipeline builds it and runs the same reference checks as for the paper, and refuses the result if a cited key is missing, an arXiv title does not match, or an identifier is unknown; on a refusal the editor is called once more with the findings and told to fix them in place. Two failed attempts end in `blocked`. The call has `edit_seconds`.

Words and endings. The two judges’ words, and where each phase can end:

phase	decisions of its judge	endings
research	<code>DRAFT</code> , <code>REVISE</code> , <code>ITERATE</code> , <code>PAUSE</code>	<code>DRAFT</code> , <code>PAUSE</code> , <code>PAUSE-ON-ITERATE</code> , <code>PAUSE-ON-REVISE</code>
paper	<code>ACCEPT</code> , <code>AMEND</code>	<code>ACCEPTED</code> , <code>PAUSE-ON-AMEND</code> , <code>blocked</code>
readable note	none	<code>DONE</code> , <code>blocked</code>

Every `PAUSE-ON-X` means “the loop named `X` ran out of its budget with the judge still asking”; the owner can raise the budget and resume from the recorded stage. `stopped` and `BLOCKED` are not endings. `stopped` is a drain checkpoint (a stop marker, a transport failure, a kill) and resumes at the recorded stage on the next start. `BLOCKED` is an unreadable decision or a missing output: the reply is saved beside the position, the runner never re-admits the thread, and `reconcile` names the one safe action.

Between transitions the pipeline itself, with no model, does the checks that gate the next state: it builds the $\text{\LaTeX}$, verifies the references, records digests, and writes the position. The verifier’s and reviewer’s judgements are the only decisions; every other arrow is mechanical or the owner’s.

4 The state of a thread

One research thread is one instance of the machine; the campaign is a set of instances plus a scheduler with a purse. Three kinds of actor touch a thread. *Agents* are model calls with a prompt and a bounded view of the thread; among them two *judges*, the verifier of a note and the reviewer of a paper, make the only decisions. The *pipeline* is code: it builds, checks, records and moves the position, and never decides. The *owner* is whoever types in the terminal, a human or a delegated session; the owner raises budgets, edits by hand and settles what is paused. Everything an agent sees or writes is a file under the thread's directory, which is what makes the record auditable after the fact.

Position. `stage`, where the machine is; `status`, one of `running`, `stopped`, `BLOCKED` or a terminal word; the counters `round` (research rounds used), `repairs` (note repairs used), the paper's `round` and the editor's `attempt`.

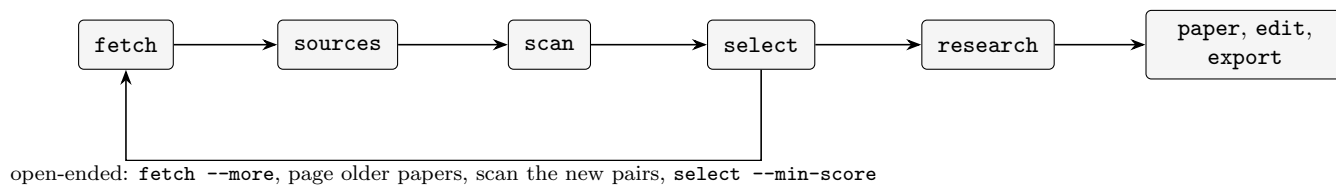
Memory, all files under `threads/<pair>/`:

- `inputs/`: the two papers, flattened `TEX` or `PDF` text, and their metadata; the seed `c`, the pair's row of `scan.jsonl`, is read from the campaign level.
- `ledger.jsonl`: append-only attributed entries of kind `idea`, `finding`, `objection`, `correction`, `intention`, `ready` or `review`; the researchers' shared memory and the verifier's evidence.
- one directory per researcher (`ada/`, `emmy/`, ...): that agent's derivations, scripts and checks. The researchers are a campaign setting (`peers`); two by default, three were tested.
- `<pair>.tex`: the consolidated note; `<pair>.verdict.json`: every verdict with the digest of the note it judged.
- `paper/`: the paper, its bibliography, its search record, the pipeline's checks per round, and every review with the digest of the paper it judged.
- `edited/`: the readable note (`note.tex`, `references.bib`, `note.pdf`).
- `status.json`, `paper/paper.json`, `edited/edit.json`: the position of each phase; `lock`: the pid of the runner working the thread.

The ledger. The ledger is the one piece of state several agents write, and its rules are what make the researchers a group rather than a set of monologues. Entries are appended through a helper, never edited; each carries a sequence number, an actor, a kind and a time, and every agent reads the whole ledger before it speaks. A *ready* entry is a claim about the ledger as it stands: it is void as soon as anyone adds a substantive entry (anything but `ready` or `review`) after it, so a researcher cannot declare itself done and walk away from an objection, and a researcher whose `ready` stands has no reason to spend another call until a partner writes. The verifier writes its verdict into the ledger as a `review` entry, so the next round's researchers, and the consolidator on a repair, read the judge's reasons where they read everything else.

Campaign level. `fetch.json` (the two queries and how far each side has been paged), `Q.jsonl` and `P.jsonl`, `scan.jsonl`, `shortlist.json`, `receipts.jsonl` (the only spend figure), and the markers `stop.json` and `health.json`.

5 The campaign



`scan.jsonl` is append-only, so a longer corpus only adds rows; a threshold cut re-selects over all rows and keeps every pair that has a thread. `explore` runs the loop `fetch --more`, `scan`, `select --min-score` for a number of passes. The runner then admits whatever is NEW on the shortlist, so research can run while the corpus grows.

6 Budgets, and what exhausting them means

budget	scope	when exhausted
<code>peer_calls</code> , <code>peer_seconds</code>	one peer round	consolidate on the ledger as it stands
<code>rounds</code>	the ITERATE loop	PAUSE-ON-ITERATE
<code>repairs</code>	the REVISE loop	PAUSE-ON-REVISE
<code>paper_rounds</code>	the AMEND loop	PAUSE-ON-AMEND
editor attempts	the build and reference checks	blocked
<code>budget_usd</code>	the campaign, at admission	stop marker; threads in flight finish; NEW pairs wait

Every model call appends a receipt, so the campaign budget is checked against the sum of receipts, never against an estimate alone. A call that times out with no usage is charged at `call_estimate_usd`. Owner rounds are not budgeted: the owner is the budget.

7 Interrupts and hazards

- A stop marker (operator, guard, or first Ctrl-C) stops admission; calls in flight land and write their checkpoints; the runner exits; the next start resumes each pair at its recorded stage.
- No session within the grace time is a transport failure: no receipt, the pair is **stopped**, two in a row set the health marker and pause admission until a probe succeeds. The `paper` and `edit` commands skip the failed pair and continue.
- `reconcile` reads the position of one pair and names the one safe action: start, resume peers, run consolidate, run verify, or nothing; `--apply` performs it.
- The lock protects a thread from two runners. It does not protect the paper or editor stage from two concurrent commands on the same pair; that is an operator error the code does not yet catch.

A Agent resources against human resources, per pair

The campaign. The reference campaign ran on September 11-13 2026. The two corpora were the ten most recent arXiv papers for the queries “mechanism design” (*Q*, dated 28 August to 4 September) and “agentic cooperation” (*P*, 16 July to 3 September). A Claude Sonnet judge scanned all 100 pairs from titles and abstracts, giving each a feasibility and a gain on a scale of 0 to 100 and

a one-paragraph connexion that serves as a seed; the score is their product, and the 14 pairs at or above 1500 were selected. Each thread ran two Opus 5 researcher agents with web search. The caps were **rounds** 4, **repairs** 1 and **paper rounds** 3, under a campaign budget of 200 USD. Outcomes: 5 threads ended DRAFT, 8 PAUSE and 1 PAUSE-ON-ITERATE; the 5 drafts became papers, 4 accepted by the reviewer within the run and 1 after the owner’s edits; all 14 threads got a readable note. Total: 330 model calls and 187 USD at API prices.

Agent figures are taken from the receipts of that campaign, as medians over threads. Means are dragged up by three threads that hit time-outs and by one paper whose seven review rounds were spread over two days. Human figures are estimates for a postdoc-level researcher at a fully loaded cost of about 60 USD per hour, stated so that they can be rescaled.

agent, per pair	calls	USD	wall time	notes
scan	1	0.03	11 s	100 pairs for 3.34 USD
research (peers, consolidate, verify)	12	6.8	8 min	mean 9.8 USD and 51 min; worst thread 30 USD and 6 h
paper (author, review)	4 to 11	3.5	5 min per round	5 pairs; the worst took 14.7 USD over 7 rounds
readable note (editor)	1	1.0	2 min	
pair to reviewed paper	about 20	about 12	about 20 min	about 8 USD and 10 min for a pair that pauses

A second run of eight pairs with three peers on the Codex model gave a research stage of 7.8 USD and 7 minutes per pair: the model choice moves the cost little, the peer count moves the time.

human, per pair, same deliverables	hours	USD	calendar
scan two abstracts and decide	0.25	15	minutes
read both papers, derive, write the note	25 to 40	1,500 to 2,400	3 to 5 days
a colleague verifies the note	2 to 4	120 to 240	when free
paper with literature search	60 to 80	3,600 to 4,800	2 weeks
a referee’s review	4 to 8	240 to 480	weeks of latency
readable write-up for outsiders	4	240	half a day
pair to reviewed paper	about 100	about 6,000	3 to 4 weeks

Per pair the agent pipeline is therefore roughly 500 times cheaper and 100 times faster on the calendar. At the scale of a campaign the gap is what makes the method exist at all: scanning 100 pairs costs 3 USD against 25 human hours, and the 14 threads and 5 papers of the reference campaign cost 175 USD, about three loaded postdoc days, against something like a working year.

Three caveats keep the comparison fair.

- *Not the same quality.* The accepted papers are correct technical remarks, the kind a good student writes in a week, not the pair a senior researcher would choose nor the paper they would write. A human would also never look at 100 pairs; taste replaces breadth.
- *Human time does not go to zero.* The owner still reads the readable notes, clears the decision queue and settles the paused pairs: on this campaign of the order of 10 to 20 minutes per pair, plus an hour of attention and about 12 USD for the one paper that was rescued by hand. The pipeline moves human effort from producing to choosing and reading.
- *Wall time is set by the slowest call, not by the sum.* Peers run concurrently, so a thread’s clock is its longest call and its time-outs; the 8-minute median hides a 6-hour worst case.